



Smart Textile Manufacturing Quality Inspection and Production Optimization Platform

CI/CD PIPELINE DESIGN EXERCISE

Submitted in the partial fulfilment for the Course of

CSA1016

Software Engineering

to the award of the degree of

BACHELOR OF ENGINEERING

IN

INFORMATION TECHNOLOGY

Submitted by

B Priyanka (192421206)

Under the supervision of

Dr. K. Anithadavamani

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai - 602105

August 2026

1. INTRODUCTION AND CI/CD REQUIREMENTS

1.1 Introduction

The **Smart Textile Manufacturing Quality Inspection and Production Optimization Platform** is designed to support textile manufacturing through fabric defect detection, machine monitoring, production analysis, alerts, reporting, and data management. Since the system contains multiple software components and requires regular updates, a reliable CI/CD process is required to automate software integration, testing, quality checking, packaging, and deployment.

Continuous Integration (CI) ensures that code changes are regularly integrated into the shared repository and automatically tested. Continuous Deployment (CD) automates the delivery of validated software to testing and production environments.

The proposed CI/CD pipeline provides a structured process from code commit to deployment and monitoring. It helps reduce manual errors, identify defects early, improve software quality, and support reliable releases.

1.2 CI/CD Requirements

The proposed system requires the following CI/CD capabilities:

- Centralized source code management.
- Automatic pipeline execution after code changes.
- Automated application build.
- Automated unit and integration testing.
- API testing for backend services.
- Code quality analysis.
- Security and vulnerability scanning.
- Docker-based application packaging.
- Deployment to testing and production environments.
- Environment-specific configuration.
- Notifications for successful or failed pipeline stages.
- Monitoring after deployment.
- Rollback support when a production deployment fails.
- Automated deployment approval and release control.
- Versioned builds and artifacts for easy release tracking and rollback.
- Automated logging and reporting of pipeline execution results.
- Consistent deployment across development, staging, and production environments.

2. CI/CD PIPELINE ARCHITECTURE AND WORKFLOW

The proposed CI/CD pipeline follows a sequence of automated stages from source code management to production deployment.

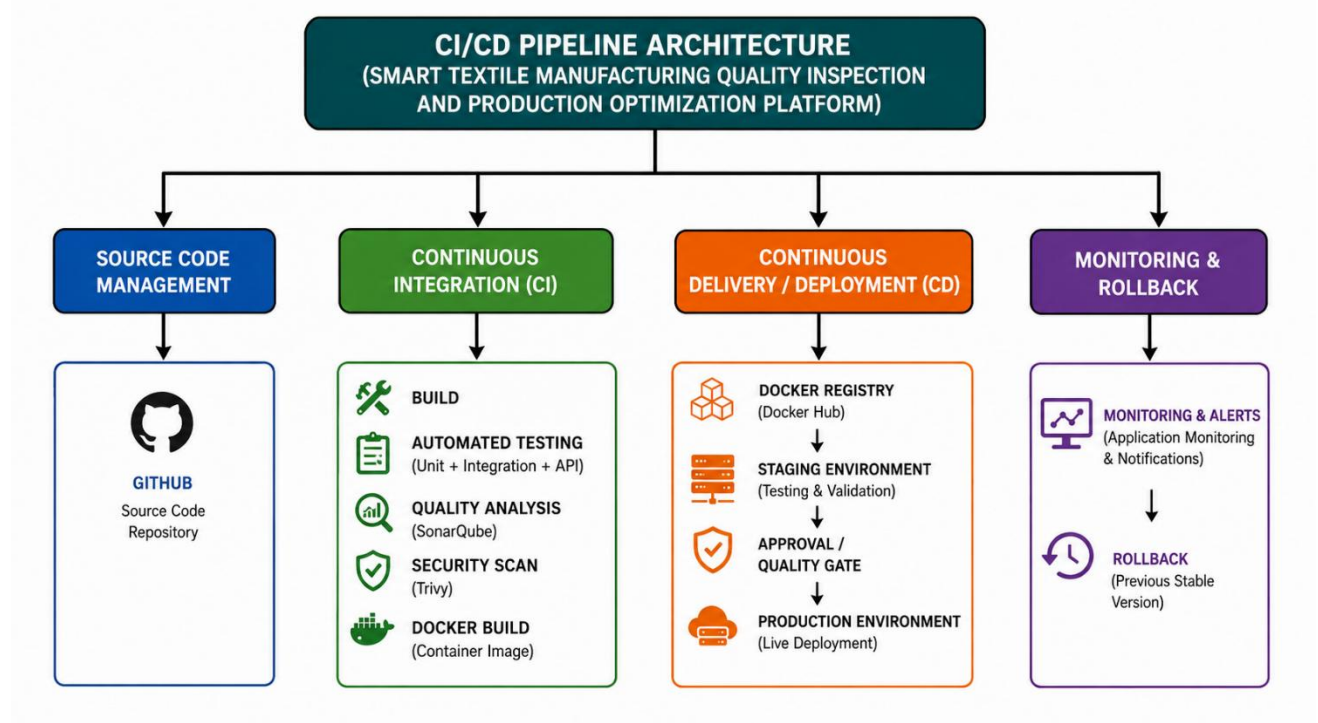


Figure 1 – CI/CD Pipeline Architecture for the Smart Textile Manufacturing Platform

The pipeline consists of the following stages:

Developer → GitHub → Jenkins → Build → Test → Quality & Security → Docker Package → Container Registry → Staging → Approval → Production → Monitoring

The source code is maintained in GitHub. Jenkins automatically starts the pipeline when a new change is committed. The application is built and tested before code-quality and security checks are performed. Each stage proceeds only after the previous stage succeeds, while failures stop the pipeline for correction.

After successful validation, a Docker image is created and stored in a container registry. The image is deployed to staging for verification and, after approval, promoted to production.

Monitoring is performed after deployment to identify errors and availability issues. Jenkins records pipeline results and provides notifications for failures or successful operations. If a critical problem occurs, the previous stable version can be restored using the rollback mechanism, providing a reliable and controlled delivery process.

2.1 Pipeline Stages

Stage	Purpose
Source	Retrieve the latest source code from GitHub
Build	Install dependencies and build the application
Unit Test	Verify individual application components
Integration/API Test	Verify interaction between application components and APIs
Quality Analysis	Check code quality and maintainability
Security Scan	Detect vulnerabilities and insecure dependencies
Package	Create a Docker image of the application
Registry	Store the validated Docker image
Staging	Deploy and validate the application in a testing environment
Approval	Verify readiness for production deployment
Production	Deploy the approved version
Monitoring	Monitor application health and deployment status
Rollback	Restore the previous stable version if required

3. TOOLS AND TECHNOLOGY SELECTION

3.1 Source Code Management

GitHub is selected for source code management. It provides centralized repository management, version control, branching, pull requests, and collaboration.

Git is used to track source-code changes and maintain different versions of the application.

3.2 CI/CD Automation

Jenkins is selected as the CI/CD automation server. Jenkins can automatically execute the pipeline whenever changes are pushed to the repository.

It can coordinate building, testing, quality analysis, Docker packaging, and deployment stages.

3.3 Application Build

The appropriate build commands are executed according to the application components. Dependencies are installed automatically during the build stage.

A failed build stops the pipeline and prevents defective code from progressing to later stages.

3.4 Automated Testing

Pytest can be used for automated Python unit testing. API testing can be performed using **Postman/Newman** to verify backend services and API responses.

Automated tests are executed before deployment so that application defects can be detected early.

3.5 Code Quality Analysis

SonarQube can be integrated with Jenkins to analyse source code quality.

It can identify issues such as code smells, duplicated code, bugs, and maintainability problems.

The pipeline can stop when the defined quality gate is not satisfied.

3.6 Security Scanning

Trivy can be used to scan Docker images and dependencies for known vulnerabilities.

Security scanning is performed before deployment so that vulnerable application packages or container components are not unnecessarily promoted to production.

3.7 Containerization

Docker is used to package the application and its dependencies into a consistent container image.

Containerization reduces differences between development, staging, and production environments.

3.8 Container Registry

A container registry such as **Docker Hub** can be used to store versioned Docker images.

Each successful build can generate a uniquely identifiable image tag, allowing specific application versions to be deployed or restored.

3.9 Tool Selection Summary

Pipeline Activity	Selected Tool	Purpose
Source Control	Git + GitHub	Version control and collaboration
CI/CD	Jenkins	Pipeline automation
Unit Testing	Pytest	Automated application testing
API Testing	Postman/Newman	API validation
Quality Analysis	SonarQube	Code quality checking
Security Scanning	Trivy	Vulnerability detection
Containerization	Docker	Application packaging
Container Registry	Docker Hub	Docker image storage
Deployment	Docker/Deployment Server	Application deployment
Notifications	Jenkins	Pipeline status notifications

4. AUTOMATED TESTING AND QUALITY INTEGRATION

Automated testing is integrated into the CI/CD pipeline to identify defects before deployment. A failed test or quality check prevents the application from moving to the next stage.

4.1 Unit Testing

Unit tests verify individual functions and components independently.

For example, functions related to user authentication, production calculations, defect data processing, and report generation can be tested individually.

4.2 Integration Testing

Integration tests verify communication between different components such as the application, database, machine-monitoring services, and analytics services.

The purpose is to identify errors that may not appear during individual unit testing.

4.3 API Testing

API tests verify important backend operations such as authentication, machine data retrieval, production data submission, defect inspection results, alert generation, and report retrieval.

API testing ensures that services return the expected status codes and responses.

4.4 Code Quality Testing

After functional testing, SonarQube performs code-quality analysis. The pipeline checks the configured quality gate before allowing the application to continue.

4.5 Security Testing

Security scanning is performed on dependencies and Docker images. Vulnerabilities above the configured severity level can cause the pipeline to fail.

4.6 Testing Flow

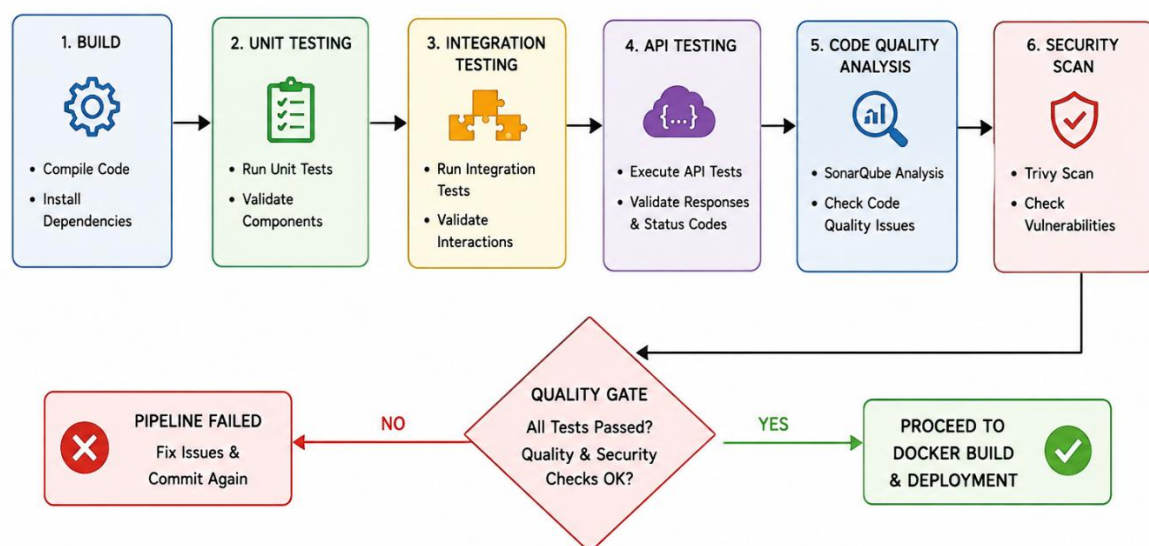


Figure 2 – Automated Testing and Quality Gate Flow

The testing sequence is:

Build → Unit Tests → Integration Tests → API Tests → Code Quality → Security Scan → Quality Gate

If any critical stage fails, the pipeline stops and the build is not deployed.

5. DEPLOYMENT STRATEGY

The proposed system uses separate environments to reduce deployment risks.

5.1 Development Environment

The development environment is used by developers to implement and test new features.

Code changes are committed to the GitHub repository and automatically trigger the CI pipeline.

5.2 Testing/Staging Environment

The staging environment provides an environment similar to production where the application can be validated before release.

After successful build, testing, quality analysis, and security scanning, the Docker image is deployed to staging.

Functional and integration verification is performed before production approval.

5.3 Production Environment

The production environment contains the live application used by the textile manufacturing organization.

Only successfully validated and approved versions are deployed to production.

5.4 Deployment Flow

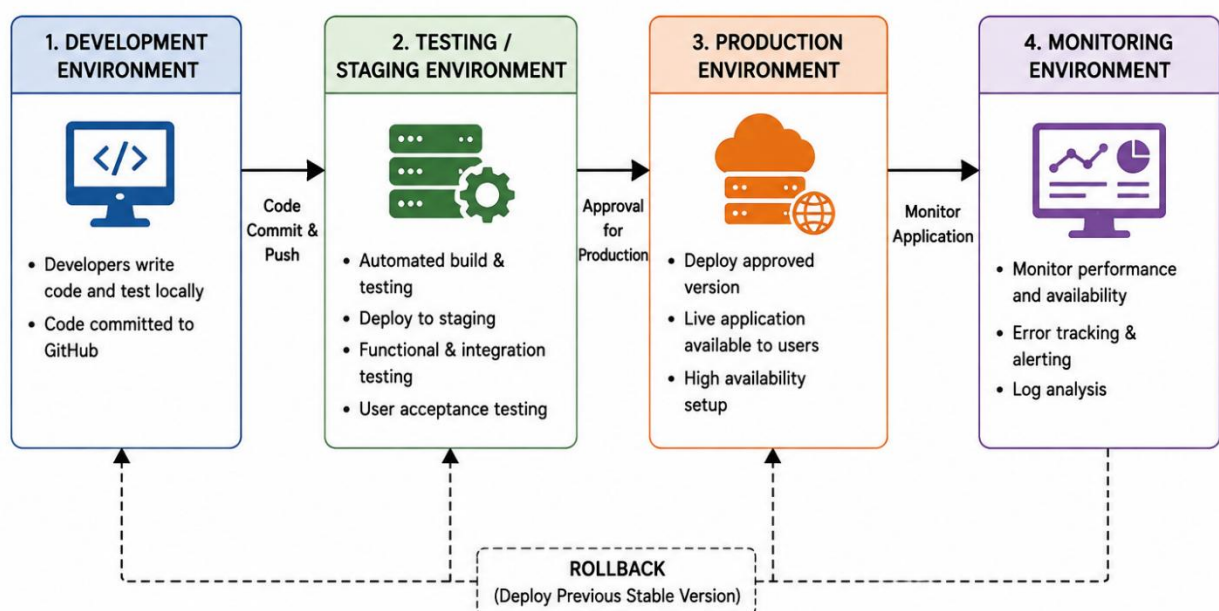


Figure 3 – CI/CD Deployment Environment Flow

Development



Automated Build & Testing



Testing / Staging



Validation & Approval



Production



Monitoring

This separation reduces the risk of directly deploying untested changes to the production environment.

5.5 Deployment Strategy Summary

Environment	Purpose	Deployment Condition
Development	Feature development	Developer changes
Testing/Staging	Validation and testing	CI pipeline success
Production	Live application	Approval after successful staging

6. SECURITY, MONITORING AND ROLLBACK STRATEGY

6.1 Security

Security is incorporated into different stages of the pipeline rather than being performed only after deployment.

The following practices are used:

- Repository access is controlled using authentication and permissions.
- Sensitive credentials are stored as protected Jenkins credentials.
- Secrets are not stored directly in source code.
- Dependencies are checked for known vulnerabilities.
- Docker images are scanned before deployment.
- Production deployment permissions are restricted.
- Only validated images are promoted to production.

6.2 Quality Gates

Quality gates are used to prevent defective or vulnerable builds from reaching production.

The pipeline can stop when:

- Unit tests fail.
- Integration tests fail.
- API tests fail.
- Code quality requirements are not satisfied.
- Critical security vulnerabilities are detected.
- Docker image creation fails.

6.3 Notifications

Jenkins can send notifications when important pipeline events occur.

Notifications can be generated for:

- Successful pipeline execution.
- Failed builds.
- Failed tests.
- Security or quality gate failures.
- Successful staging deployment.
- Production deployment.
- Rollback events.

6.4 Monitoring

After production deployment, the application should be monitored for availability and operational problems.

Important monitoring indicators include:

- Application availability.
- Container health.
- API response status.
- Application errors.
- Resource utilization.
- Production service availability.

6.5 Rollback Strategy

If a newly deployed version causes a critical problem, the system can restore the previous stable Docker image. The rollback process is:

Problem Detected → Stop New Version → Select Previous Stable Image → Deploy Previous Version → Verify Application → Continue Monitoring

7. CI/CD WORKFLOW EXPLANATION

The complete CI/CD workflow begins when a developer commits code changes to the GitHub repository.

Step 1 – Code Commit

The developer pushes the latest code to GitHub.

Step 2 – Pipeline Trigger

Jenkins detects the new change and automatically starts the CI/CD pipeline.

Step 3 – Build

Jenkins retrieves the source code and installs the required dependencies. The application is built and checked for build errors.

Step 4 – Automated Testing

Unit tests, integration tests, and API tests are executed. If a critical test fails, the pipeline stops.

Step 5 – Quality Analysis

SonarQube analyses the source code and evaluates the configured quality gate.

Step 6 – Security Scanning

Dependencies and the application container are scanned for known vulnerabilities.

Step 7 – Docker Packaging

A Docker image is created containing the application and required dependencies.

Step 8 – Registry

The validated Docker image is tagged with a version and stored in the container registry.

Step 9 – Staging Deployment

The image is deployed to the staging environment. Additional validation is performed to confirm that the application operates correctly.

Step 10 – Production Approval

After successful staging validation, the release is approved for production deployment.

Step 11 – Production Deployment

The approved Docker image is deployed to the production environment.

Step 12 – Monitoring and Rollback

The deployed application is monitored. If a critical failure occurs, the previous stable version is restored through the rollback process.

8. CONCLUSION

The proposed CI/CD pipeline for the **Smart Textile Manufacturing Quality Inspection and Production Optimization Platform** provides an automated process for integrating, testing, packaging, and deploying software.

The pipeline uses GitHub for source-code management, Jenkins for CI/CD automation, automated testing tools for functional verification, SonarQube for code-quality analysis, Trivy for security scanning, and Docker for consistent application packaging.

The separation of development, staging, and production environments reduces deployment risks. Automated testing and quality gates prevent defective builds from progressing, while security scanning helps identify vulnerabilities before deployment.

The use of versioned container images provides a practical rollback mechanism when a production deployment causes problems. Monitoring and notifications also provide visibility into pipeline and application status.

Overall, the proposed CI/CD design supports **faster delivery, improved software quality, security, reliability, and controlled deployment** of the Smart Textile Manufacturing platform.